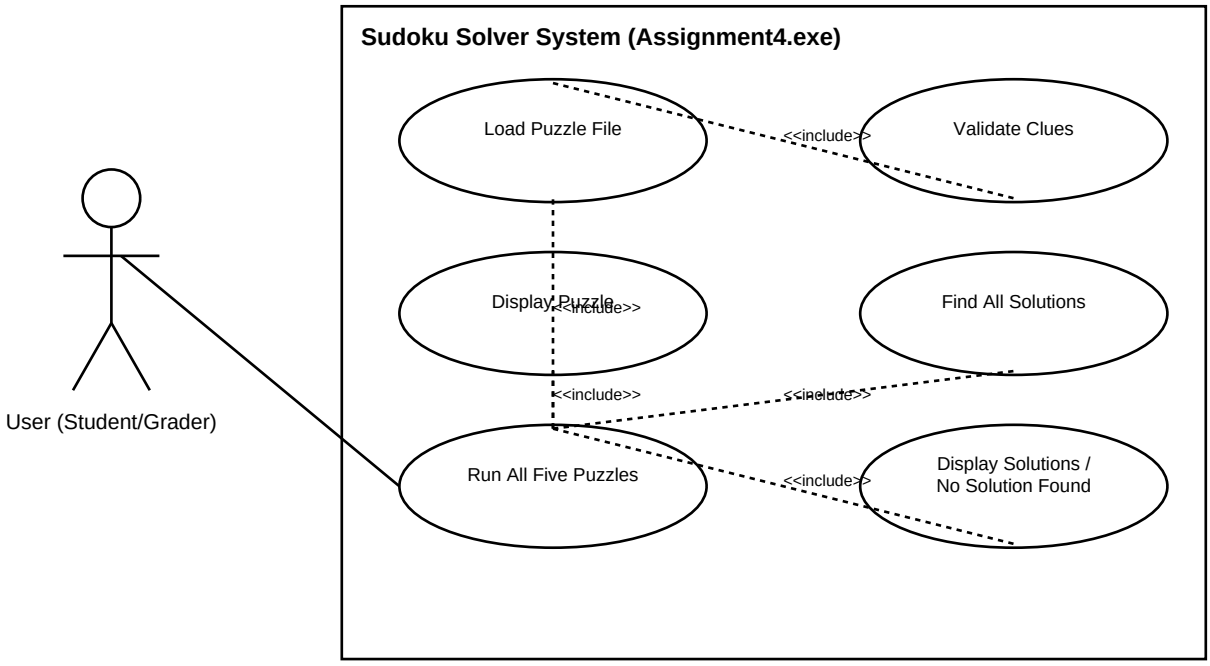
Assignment 4 - UML Use Case Model: Solve Sudoku Puzzles

Created with the help of Claude (Anthropic GenAI), October 12, 2026. Author: Logan Sigg

a. Use case identified

UC-1: Solve Sudoku Puzzle Files (primary actor: User; the grader or student runs the program).

Use case diagram



b. Brief description

The user runs the program. For each of puzzle1.txt through puzzle5.txt the system reads the puzzle, prints it, searches exhaustively using recursive backtracking, and prints every solution found, or 'No solution found' when none exists.

c. Step-by-step flow of events

1. User runs Assignment4.exe from the directory containing the puzzle files.
2. System opens the next puzzle file (puzzle1.txt ... puzzle5.txt).
3. System reads 81 cells (digit or '_') and validates them.
4. System prints the file name and the puzzle.
5. System searches recursively for all solutions.
6. System prints the number of solutions and each solution grid, or 'No solution found'.
7. Steps 2-6 repeat until all five files are processed; the program ends.

d. Detailed description

- **Use case name:** Solve Sudoku Puzzle Files
- **Actor:** User (student / GTA)

- **Goal:** See the solution(s) of all five puzzle files
- **Preconditions:** Assignment4.exe and puzzle1-5.txt are in the same directory; each file has 9 rows of 9 space-separated tokens
- **Trigger:** User executes the program
- **Main success scenario:** Steps 1-7 above. Each puzzle yields one or more solutions.
- **Alternate flow A - multiple solutions:** At step 6 the system finds more than one grid (e.g., puzzle5.txt has 2). It prints all of them, numbered.
- **Alternate flow B - no solution:** At step 5 the search exhausts all candidates (e.g., puzzle4.txt). System prints 'No solution found'.
- **Exception - file missing/malformed:** At step 2/3 the file cannot be opened, has a bad token, or its clues already violate a row/column/block rule. System prints an error message for that file and continues with the next file.
- **Algorithm (included behavior):** Pick the blank cell with fewest candidates; try each candidate; recurse; undo (backtrack) on failure; record a solution when no blanks remain and keep searching for more.
- **Postconditions:** All five puzzles have been reported; input files unchanged.
- **Constraints:** Brute-force depth-first search; object-oriented C++ (Board, SudokuSolver classes); runs on EECS cycle servers.